Day 1 – Lab 1.3: Deploy, Configure and Monitor Azure Functions

Module 3 · Deployment, Application Settings & Monitoring

Objective: Deploy the C# HTTP Function created in Lab 1.2 to Azure, configure application settings and monitor successful and failed executions using Application Insights.

Business scenario: Contoso Orders has completed local development of its Order Quote API. The development team must now publish it to Azure, apply a business discount through configuration and monitor API requests without changing application code every time the discount changes.

1. Prerequisites

Verify the following before proceeding:

- Lab 1.1 – Function App `fn-user01-orders` is available.
- Lab 1.2 – C# project `contoso-fn` builds successfully.
- HTTP Function `QuoteFunction` works locally.
- Azure Functions extension is installed in VS Code.
- Participant has permission to deploy code and update Function App settings.

Important: We will deploy to the existing Function App from Lab 1.1. Do not create another Function App.

2. Update the Order Quote Function

Before deployment, modify the Function to read the discount percentage from an Azure application setting.

Step 1 – Open the project

Open the `contoso-fn` project in VS Code.

Step 2 – Update `QuoteFunction.cs`

Replace the existing implementation with the following complete code:

```
None
using System.Globalization;
using System.Net;
using System.Text.Json;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Microsoft.Extensions.Logging;

namespace ContosoFn;

public class QuoteFunction
{
    private readonly ILogger<QuoteFunction> _logger;

    public QuoteFunction(ILogger<QuoteFunction>
logger)
    {
        _logger = logger;
    }
```

```
    [Function("QuoteFunction")]
    public async Task<HttpResponseData> Run(
        [HttpTrigger(
            AuthorizationLevel.Function,
            "post",
            Route = "quote")] HttpRequestData req)
    {
        JsonDocument document;

        try
        {
            document = await
JsonDocument.ParseAsync(req.Body);
        }
        catch (JsonException)
        {
            _logger.LogWarning("Invalid JSON
received.");

            var badJson =
req.CreateResponse(HttpStatusCode.BadRequest);

            await badJson.WriteAsJsonAsync(new
            {
                error = "Invalid JSON request body."
            });
```

```
            return badJson;
        }

        using (document)
        {
            var root = document.RootElement;

            if (root.ValueKind !=
JsonValueKind.Object ||
                !root.TryGetProperty("qty", out var
qtyElement) ||
                !root.TryGetProperty("unitPrice", out
var priceElement) ||
                !qtyElement.TryGetInt32(out int qty)
||
                !priceElement.TryGetDecimal(out
decimal unitPrice) ||
                qty <= 0 ||
                unitPrice < 0)
            {
                _logger.LogWarning("Invalid order
quote input.");

                var badRequest =

req.CreateResponse(HttpStatusCode.BadRequest);
```

```
                await badRequest.WriteAsJsonAsync(new
                {
                    error = "Provide a positive qty
and a non-negative unitPrice."
                });

                return badRequest;
            }

            decimal discountPercent = 0;

            var configuredDiscount =

Environment.GetEnvironmentVariable("DISCOUNT_PCT");

            if
(!string.IsNullOrWhiteSpace(configuredDiscount) &&
                decimal.TryParse(
                    configuredDiscount,
                    NumberStyles.Number,
                    CultureInfo.InvariantCulture,
                    out var parsedDiscount) &&
                parsedDiscount >= 0 &&
                parsedDiscount <= 100)
            {
                discountPercent = parsedDiscount;
            }
```

```
            else if
(!string.IsNullOrWhiteSpace(configuredDiscount))
            {
                _logger.LogWarning(
                    "Invalid DISCOUNT_PCT
configuration. Using zero discount.");
            }

            decimal subtotal = qty * unitPrice;
            decimal discountAmount =
                subtotal * discountPercent / 100;

            decimal total = subtotal -
discountAmount;

            _logger.LogInformation(
                "Order quote calculated. Quantity:
{Quantity}, Total: {Total}",
                qty,
                total);

            var response =

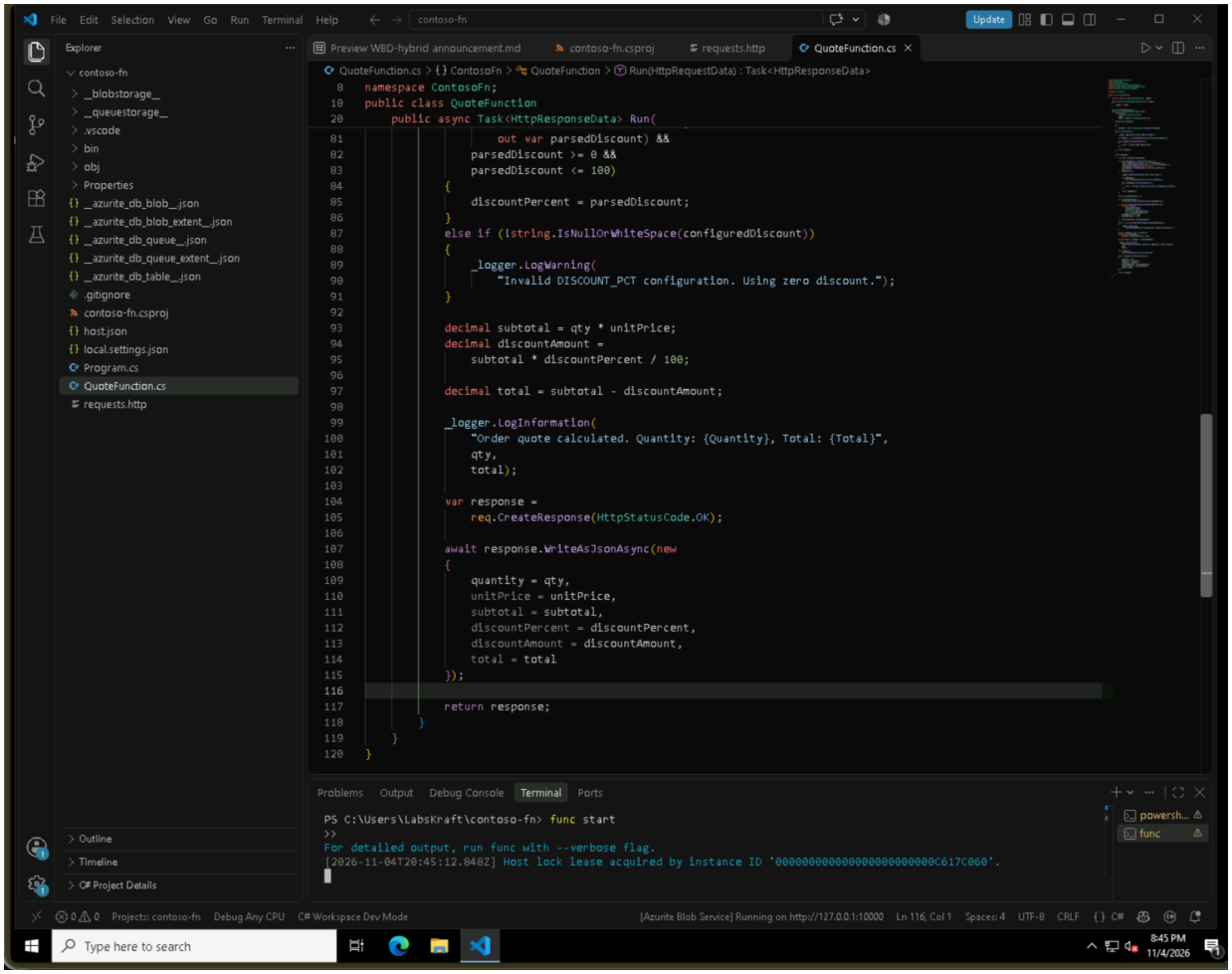
req.CreateResponse(HttpStatusCode.OK);

            await response.WriteAsJsonAsync(new
            {
```

```
                quantity = qty,
                unitPrice = unitPrice,
                subtotal = subtotal,
                discountPercent = discountPercent,
                discountAmount = discountAmount,
                total = total
            });

            return response;
        }
    }
}
```

What changed?



- Reads `DISCOUNT_PCT` from application configuration.
- Calculates discount without hardcoding the percentage.
- Logs successful calculations and invalid requests.
- Defaults to zero discount if the setting is missing or invalid.

This demonstrates how production applications separate business configuration from application code.

3. Build and Validate Locally

Open the VS Code terminal.

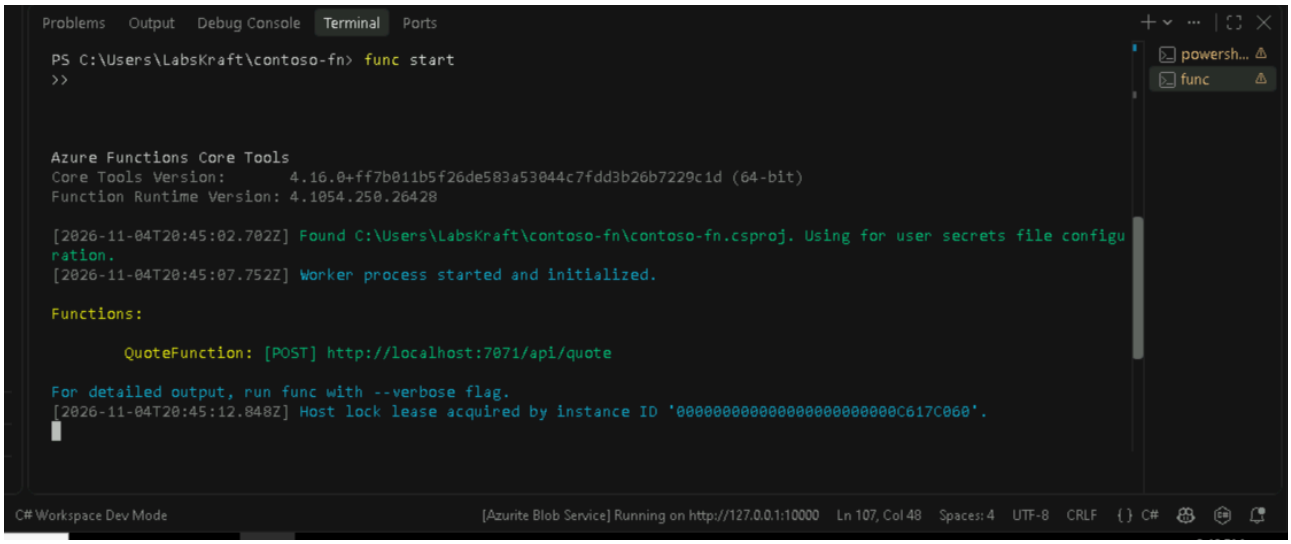
Execute:

```
dotnet build
```

Expected: Build succeeds without errors.

Start the local Function:

```
func start
```



Open `requests.http` from Lab 1.2 and send the valid request:

```
POST http://localhost:7071/api/quote
Content-Type: application/json

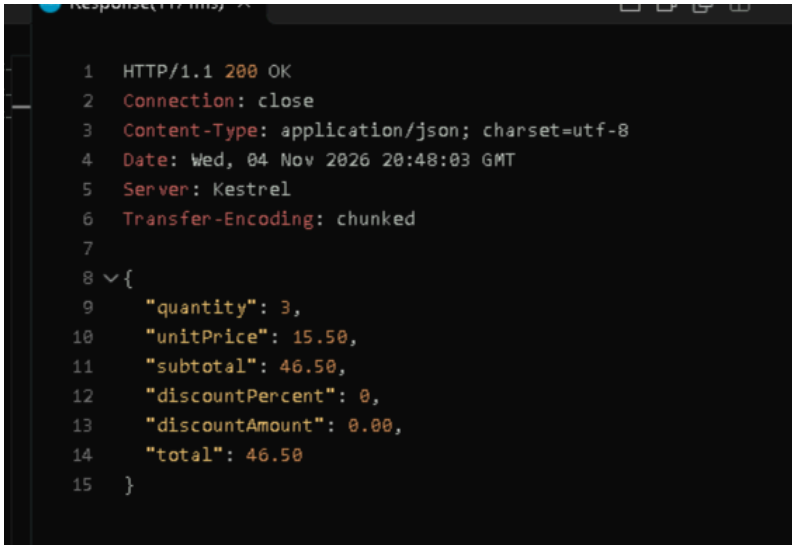
{
  "qty": 3,
  "unitPrice": 15.50
```

```
}
```

Expected response:

```
None
{
  "quantity": 3,
  "unitPrice": 15.50,
  "subtotal": 46.50,
  "discountPercent": 0,
  "discountAmount": 0,
  "total": 46.50
}
```

The local environment does not yet contain `DISCOUNT_PCT`, so the Function uses zero discount.



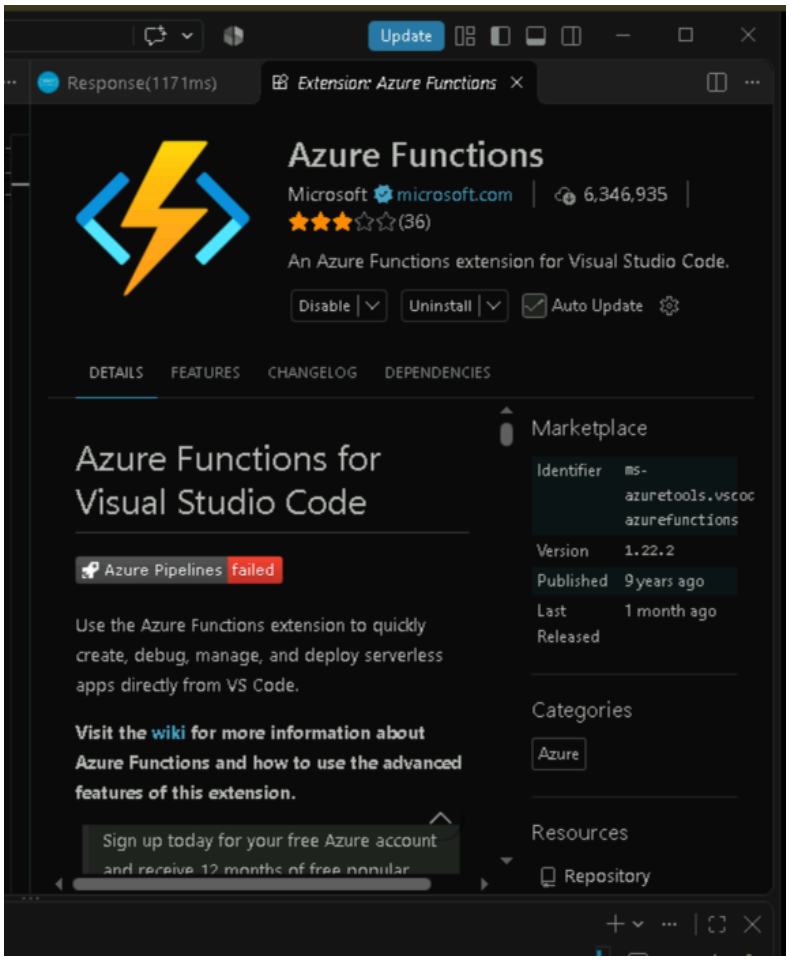
Stop the local Function host using `Ctrl + C`.

4. Deploy the Function using VS Code

We will use the VS Code Azure Functions extension rather than manually packaging the project.

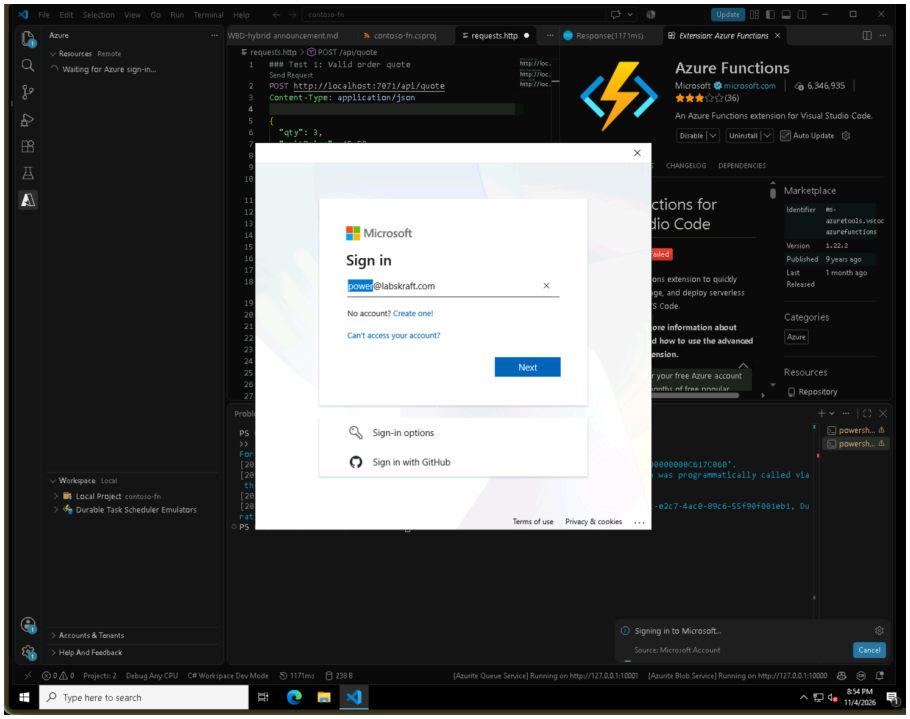
Step 1 – **Install Azure Functions VS Code Extension**

1. Click on the **Extensions** icon on the left Activity Bar (`Ctrl + Shift + X`).
2. Search for **`Azure Functions`** (by Microsoft).
3. Click **Install**.



Step 2 – Sign in to Azure

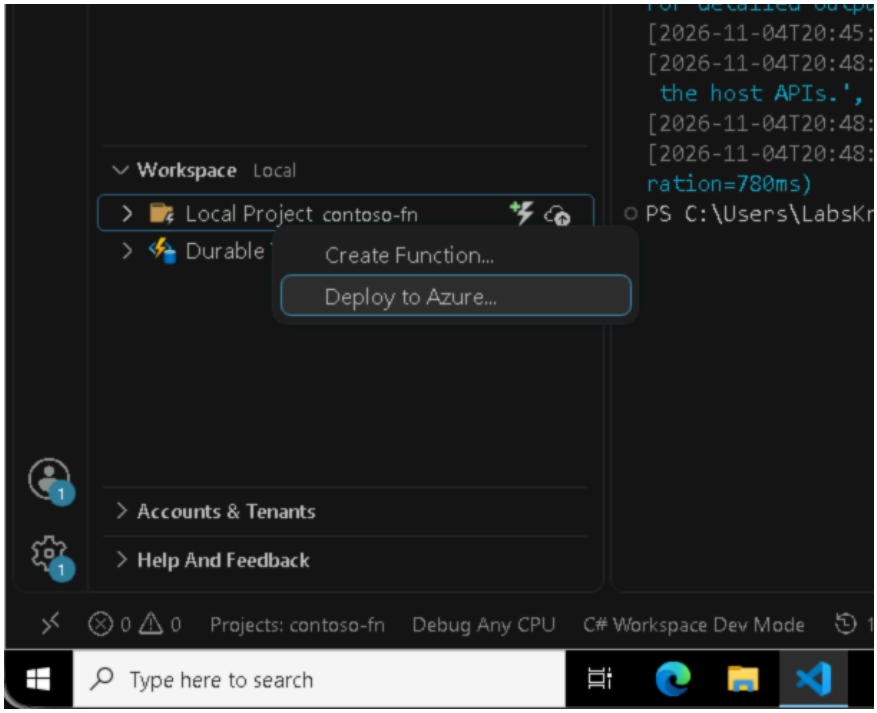
1. Open VS Code.
2. Select the Azure icon from the Activity Bar.
3. Sign in using the participant's Azure account.



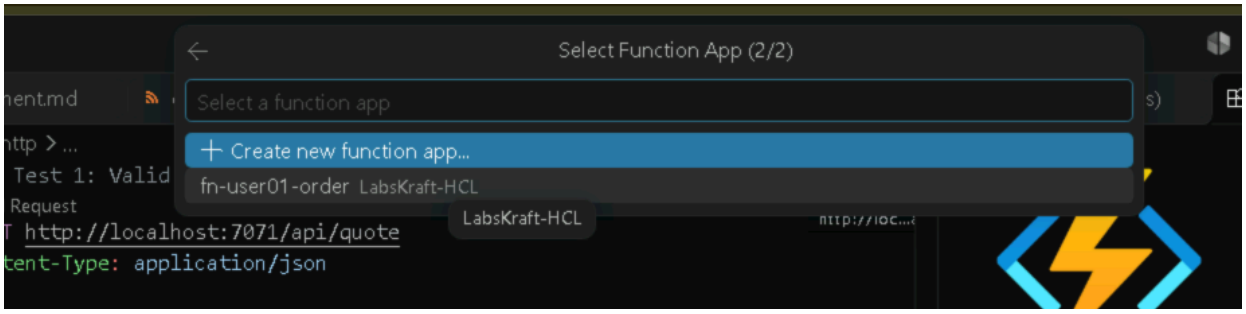
4. Expand the assigned subscription.

Step 3 – Deploy the project

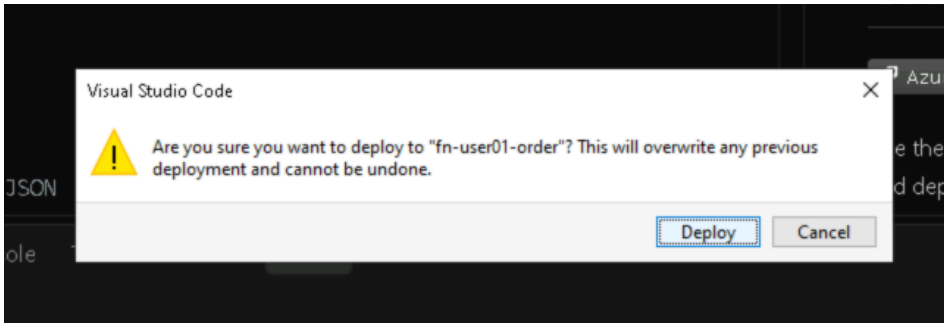
1. Open the VS Code Explorer.
2. Right-click the `contoso-fn` project folder.
3. Select Deploy to Azure.



4. Select the assigned Azure subscription.
5. Select the existing Function App:
   `fn-user01-orders`



6. Confirm the deployment when prompted.



Step 4 – Wait for deployment

VS Code will build and publish the Function project.

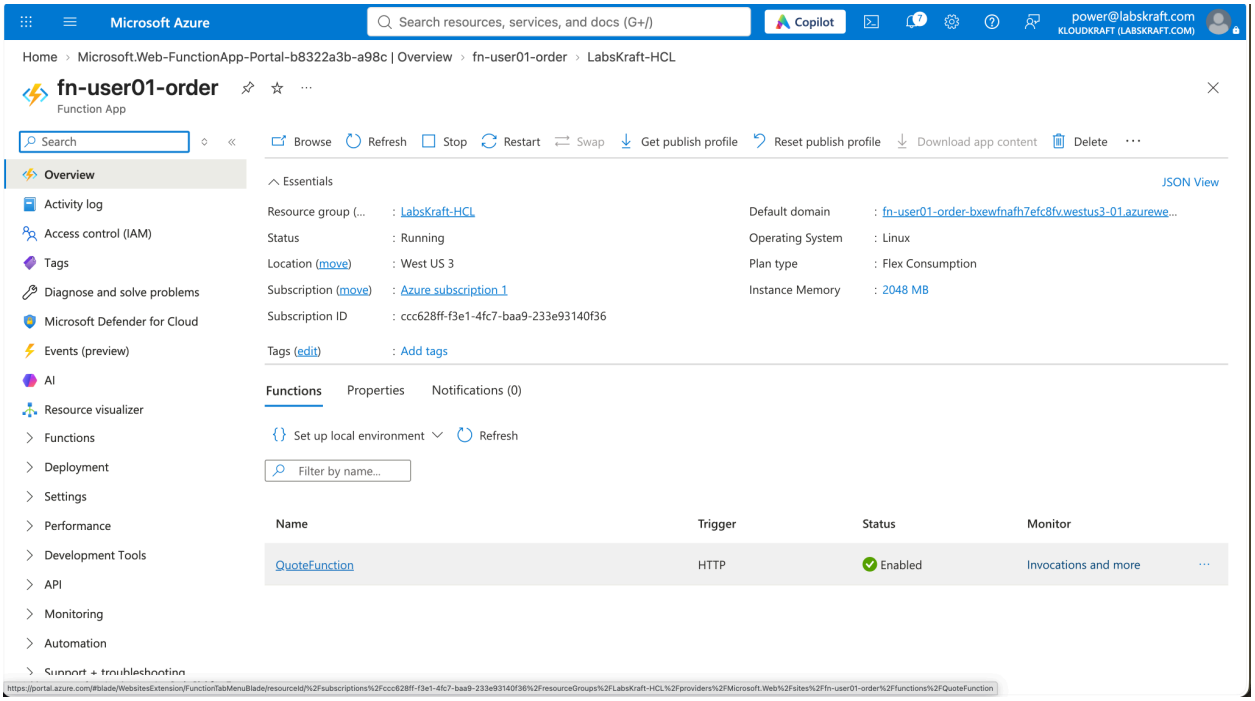
Wait until the deployment notification confirms success.

Important: If VS Code warns that existing Function App content will be overwritten, verify that you selected your own participant Function App before confirming.

Microsoft reference: [Develop Azure Functions using Visual Studio Code].

5. Verify Deployment in Azure

1. Open Azure Portal.
2. Navigate to `fn-user01-orders`.
3. Open Functions.
4. Confirm that `QuoteFunction` is listed.
5. Open the Function and inspect its endpoint URL.



Expected endpoint format:

```
https://fn-user01-orders.azurewebsites.net/api/quote
```

If the Function does not appear, check the deployment output and Function App logs before proceeding.

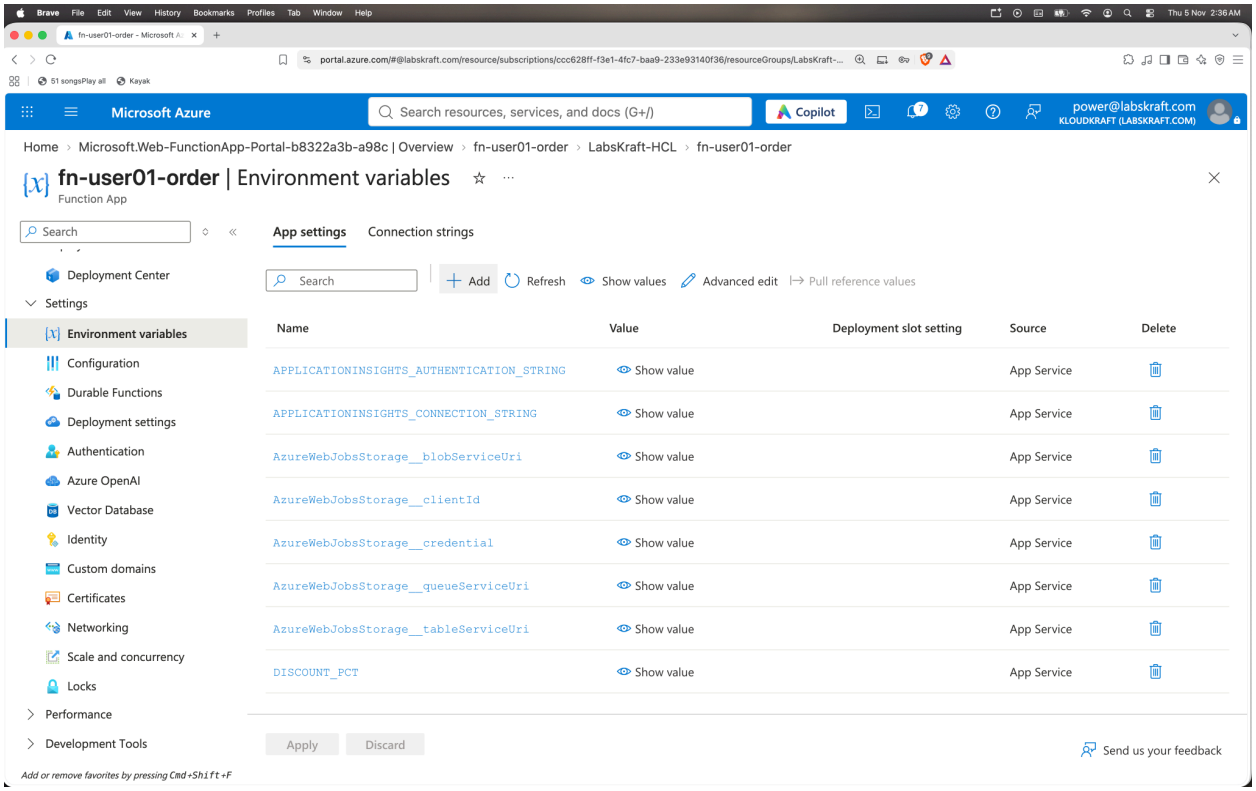
6. Configure Application Settings

Now we will configure the discount percentage without modifying the deployed code.

Step 1 – Open application settings

1. Navigate to the Function App.
2. Select Settings → Environment variables.
3. Open the application settings section.

4. Select Add.



Step 2 – Add the setting

Enter:

- Name: `DISCOUNT_PCT`
- Value: `10`

Save the configuration.

The Function App may restart automatically when application settings are saved. Wait for it to become available again.

Understanding: The Function reads the setting through `Environment.GetEnvironmentVariable()`. This allows operations teams to change application behaviour without changing the source code.

7. Test the Deployed Function

We will test the API using the deployed endpoint.

Step 1 – Retrieve the Function key

1. Open `QuoteFunction` in Azure Portal.
2. Navigate to Function Keys.
3. Copy the default Function key.

Keep this key private. Do not commit it to source code or share it in screenshots.

Step 2 – Send a valid request

You can use the REST Client extension in VS Code.

Create or update `requests.http`:

```
None
### Valid deployed request
POST
https://fn-user01-orders.azurewebsites.net/api/quote
x-functions-key: YOUR_FUNCTION_KEY
Content-Type: application/json

{
  "qty": 3,
  "unitPrice": 15.50
}
```

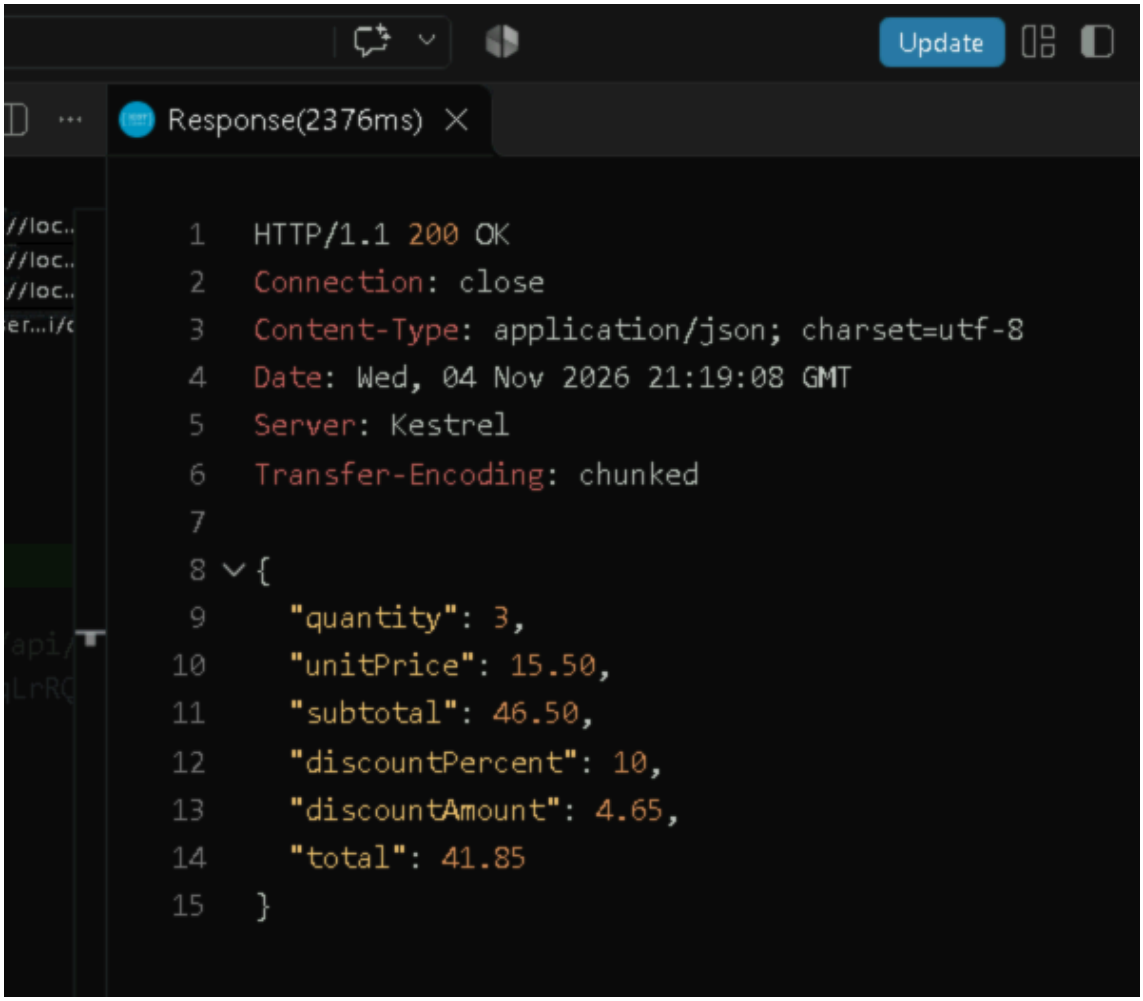
Replace `YOUR_FUNCTION_KEY` with the actual key & your function url with actual url.

Select Send Request.

Expected response:

HTTP 200 OK

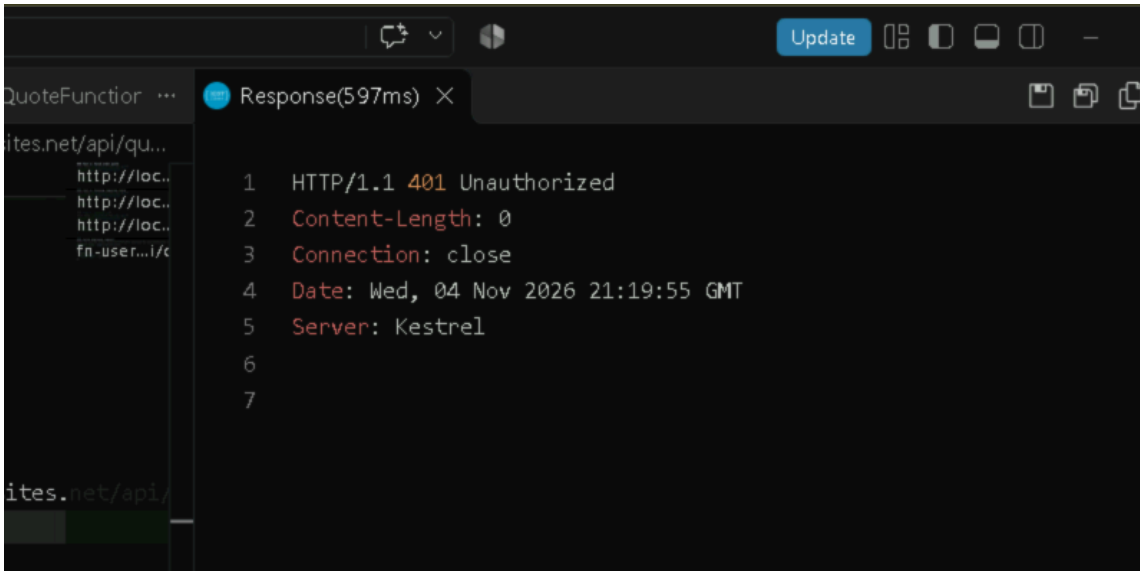
```
None
{
  "quantity": 3,
  "unitPrice": 15.50,
  "subtotal": 46.50,
  "discountPercent": 10,
  "discountAmount": 4.65,
  "total": 41.85
}
```



Step 3 – Test without the Function key

Remove the `x-functions-key` header and send the same request again.

Expected result: Azure rejects the request with an authorization error (normally HTTP 401).



Step 4 – Test invalid input

Send:

```
None
POST
https://fn-user01-orders.azurewebsites.net/api/quote
x-functions-key: YOUR_FUNCTION_KEY
Content-Type: application/json

{
  "qty": 0,
  "unitPrice": 15.50
}
```

Expected result: HTTP 400 Bad Request.

This confirms that the deployed Function performs input validation.

8. Monitor Function Execution

Now we will verify whether Azure is capturing API requests and application logs.

Step 1 – Open Application Insights

1. Navigate to the `appi-user01-orders` resource.
2. Open Investigate → Logs.
3. Select the Application Insights resource as the query scope.

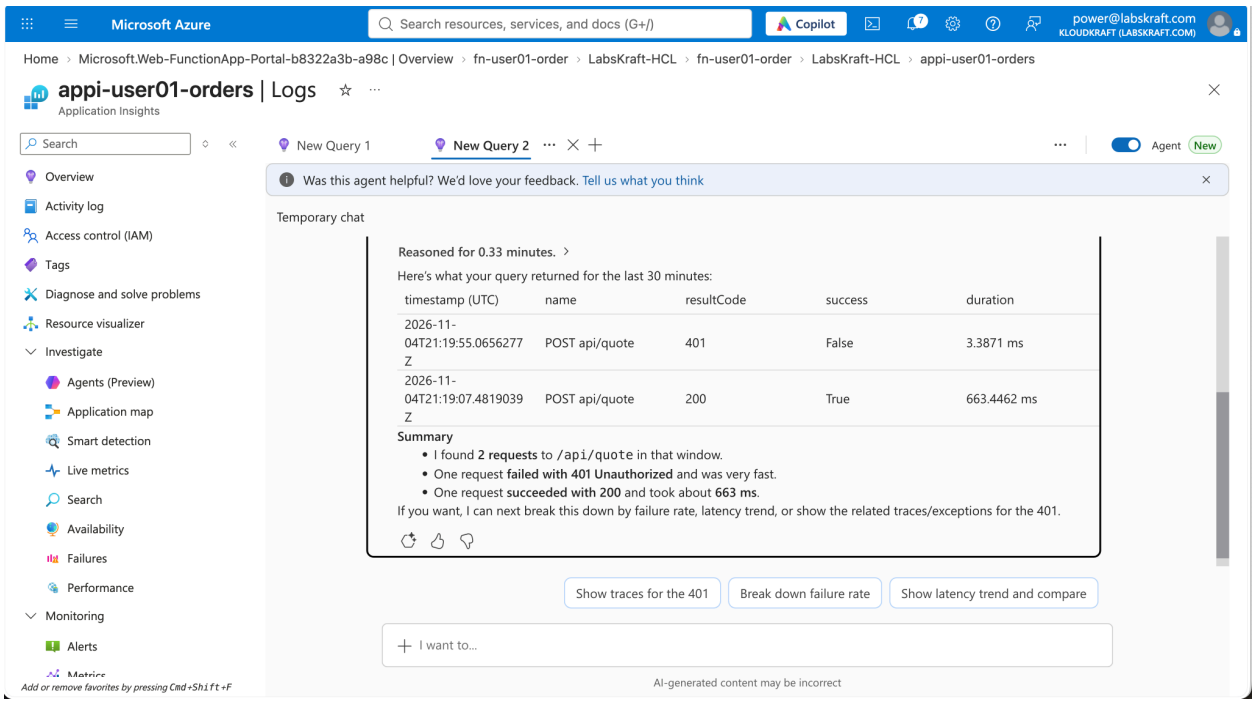
Step 2 – Inspect HTTP requests

Execute:

```
None
requests
| where timestamp > ago(30m)
| where url contains "/api/quote"
| project timestamp, name, resultCode, success,
duration
| order by timestamp desc
```

Expected observations:

- Successful request – HTTP 200.
- Invalid input – HTTP 400.
- Missing Function key – authorization failure, if the request reaches the monitored application.

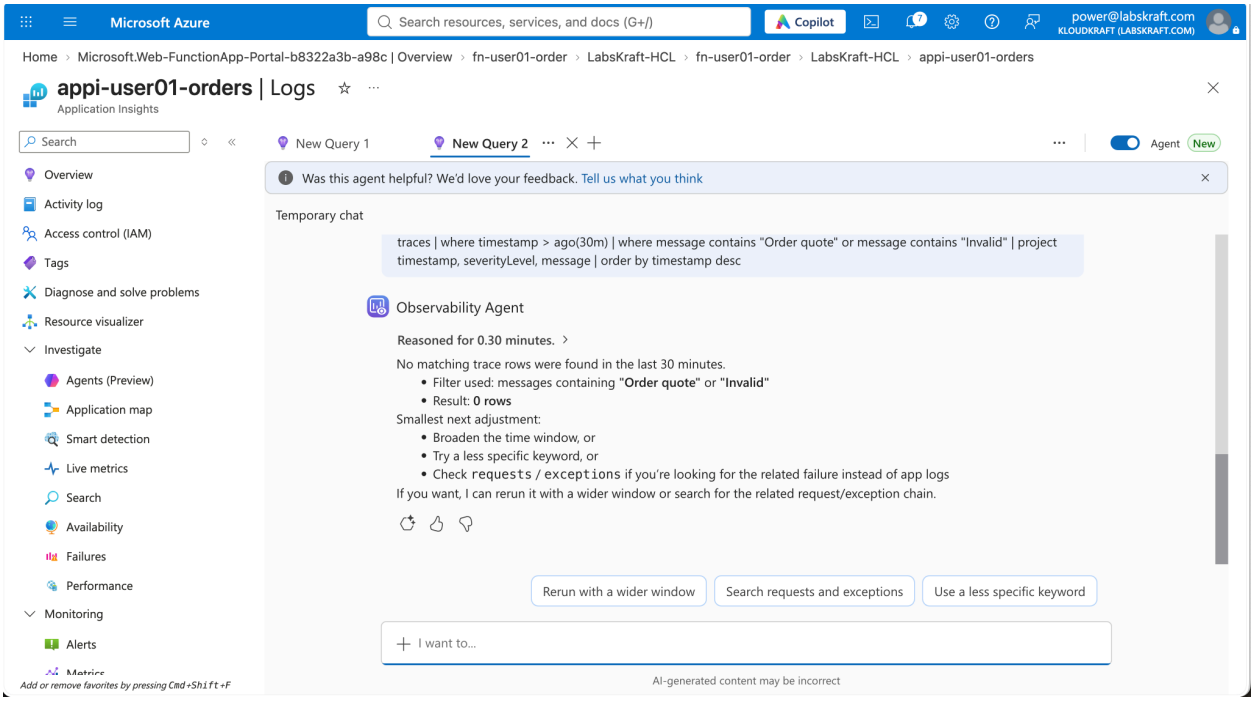


Step 3 – Inspect application traces

Execute:

None

```
traces
| where timestamp > ago(30m)
| where message contains "Order quote"
   or message contains "Invalid"
| project timestamp, severityLevel, message
| order by timestamp desc
```



Expected observations:

- Successful order quote calculation.
- Invalid JSON or invalid order input warnings.

Step 4 – Check Log Stream

1. Return to the Function App.
2. Open Monitoring → Log stream.
3. Send another valid request.
4. Observe the Function execution logs.

Allow a short delay for telemetry ingestion.

Important: Application Insights must have a valid connection string configured. If requests or traces are missing, verify the Function App's monitoring configuration, telemetry settings and logging levels.

9. Troubleshooting

Issue	Resolution
Function not visible after deployment	Check VS Code deployment output and Function host status.
HTTP 401	Verify Function key and authorization level.
HTTP 500	Inspect Function execution logs and application settings.
Discount remains zero	Verify `DISCOUNT_PCT` and restart status.
Application Insights has no data	Check connection string and telemetry configuration.
Function fails after deployment	Verify runtime configuration and deployment output.

10. Acceptance Criteria

The lab is complete when:

- C# Function project builds successfully.
- Function is deployed to the existing Azure Function App.

- `QuoteFunction` is visible in Azure Portal.
- `DISCOUNT_PCT=10` is configured.
- Valid request returns the discounted total.
- Request without a Function key is rejected.
- Invalid order input returns HTTP 400.
- Successful and failed requests are observable in Application Insights.
- Function execution logs are visible.